

NAME

sarith -- Spectrum arithmetic

USAGE

sarith input1 op input2 output

PARAMETERS**input1**

List of input images to be used as operands.

op Operator to be applied to the first operand or to both operands. The unary or single operand operators are:

- abs - absolute value
- copy - copy (see also **scopy**)
- dex - decimal exponentiation (antilog of base 10 logarithm)
- exp - base e exponentiation (antilog of natural logarithm)
- flam - convert F-nu to F-lambda
- fnu - convert F-lambda to F-nu
- inv - inverse
- ln - natural logarithm
- log - base 10 logarithm
- lum - convert magnitude to luminosity
- mag - convert luminosity to magnitude
- sqrt - square root

The binary or two operand operators are:

- replace - replace first operand values by second operand values
- + - addition
- - subtraction
- * - multiplication
- / - division
- ^ - exponentiation

input2

Lists of input spectra or constants to be used as second operands for binary operations. If a single value is specified it applies to all the first operand input images otherwise the list must match the first operand list in number.

output

List of resultant output images or root names. Image sections are ignored and if the output format is "onedspec" then any record extensions are stripped to form the root name. If no output list is specified then the input list is used and the input images are replaced by the resultant spectra. If a single output name is specified then all resultant spectra are written to the same output image or image root name. This allows packing or merging multiple spectra and requires properly setting the *clobber*, *merge*, *renumber* and *offset* parameters to achieve the desired output. If more than one output image is specified then it must match the input image list in number.

w1 = INDEF, w2 = INDEF

Starting and ending wavelengths to be copied. If w1 is not specified then the wavelength of the starting edge of the first pixel is used (wavelength at pixel coordinate 0.5) and if w2 is not specified

then the wavelength of the ending edge of the last pixel is used (wavelength of the last pixel plus 0.5). If both are not specified, that is set to INDEF, then the whole spectrum is copied and the *rebin* parameter is ignored. Note that by specifying both endpoints the copied region can be set to have increasing or decreasing wavelength per pixel. If the spectrum only partially covers the specified range only that portion of the spectrum within the range is copied. It is an error if the range is entirely outside that of a spectrum.

apertures = "", beams = ""

List of apertures and beams to be selected from the input spectra. The logical intersection of the two lists is selected. The null list selects all apertures or beams. A list consists of comma separated numbers and ranges of numbers. A range is specified by a hyphen. An optional step size may be given by 'x' followed by a number. See **xtools.ranges** for more information. If the first character is "!" then the apertures/beams not in the list are selected. Note that a "!" in either of the lists complements the intersection of the two lists. For longslit input spectra the aperture numbers selects the lines or columns to be extracted. For 3D Fabry-Perot spectra the aperture numbers select the first spatial axis.

bands = ""

List of bands in 3D multispec. For 3D spatial spectra the band parameter applies to the second spatial axis. The null list selects all bands. The syntax is as described above.

apmodulus= 0

Modulus to be applied to the input aperture numbers before matching against the aperture list. If zero then no modulus is used. This is used to select apertures which are related by the same modulus, typically a factor of 10; for example, 10, 1010, and 2010 with a related.

reverse= no

Reverse the order of the operands in a binary operation? Because the first operand is used as the image header template, dispersion coordinate template, and output image in the case of a image and not a constant. To allow certain operations, for example subtracting a spectra from a constant or using the subtractand as the dispersion coordinate template, the reverse option is used to reverse the order of the operands in a binary operation.

ignoreaps= no

Ignore aperture numbers in the second operand? Normally, spectra in binary operations must have matching aperture numbers, otherwise an error is printed. If this parameter is yes then the spectra are matched by line number with the last line being used if the second operand spectrum has fewer lines than the first operand spectrum. This is generally used to allow using a single spectrum with multiple aperture spectra.

format="multispec" (multispec|onedspec)

Output image format and name syntax. The "multispec" format consists of one or more spectra in the same image file. The "onedspec" format consists of a single spectrum per image with names having a root name and a four digit aperture number extension. Note that converting to "onedspec" format from three dimensional images where the third dimension contains associated spectra will not include data from the extra dimension. Image sections may be used in this case.

renumber= no

Renumber the output aperture numbers? If set the output aperture numbers, including any preexisting spectra when merging, are renumbered beginning with 1. The *offset* parameter may be used to change the starting number.

offset = 0

Offset to be added to the input or renumbered aperture number to form the final output aperture number.

clobber= no

Modify an existing output image either by overwriting or merging?

merge= no

Merge apertures into existing spectra? This requires that the *clobber* parameter be set. If not merging then the selected spectra entirely replace those in existing output images. If merging then the input spectra replace those in the output image with the same aperture number and new apertures are added if not present.

rebin= yes

Rebin the spectrum to the exact wavelength range specified by the *w1* and *w2* parameters? If the range is given as INDEF for both endpoints this parameter does not apply. If a range is given and this parameter is not set then the pixels in the specified range (using the nearest pixels to the endpoint wavelengths) are copied without rebinning. In this case the wavelength of the first pixel may not be exactly that specified by *w1* and the dispersion, including non-linear dispersions, is unchanged. If this parameter is set the spectra are interpolated to have the first and last pixels at exactly the specified endpoint wavelengths while preserving the same number of pixels in the interval. Linear and log-linear dispersion types are maintained while non-linear dispersions are linearized.

errval= 0.

Value for resultant pixel if an arithmetic error occurs such as dividing by zero or the square root of a negative number.

verbose= no

Print a record of each operation?

DESCRIPTION

Sarith performs arithmetic operations on spectra. It is distinguished from **imarith** in that it includes unary operators, like **imfunction** but with some specific to astronomical spectra, and binary operations between two spectra are performed in dispersion coordinate space (typically wavelength) rather than logical pixel space. In the latter case the spectra are checked for matching dispersion functions (which are not necessarily linear) and, if they don't match, the second operand is interpolated without flux conservation. (If flux conservation is desired then the task **dispcor** should be used first.) Thus, the spectra may have different dispersion functions but the arithmetic is done at matching wavelengths. The default interpolation function is a 5th order polynomial. The choice of interpolation type is made with the package parameter "interp". It may be set to "nearest", "linear", "spline3", "poly5", or "sinc". Remember that this applies to all tasks which might need to interpolate spectra in the **onedspec** and associated packages. For a discussion of interpolation types see **onedspec**.

The unary operators operate on the spectra in the first operand list to produce the specified output spectra, which may be the same as the input spectra. The operators include:

- abs - absolute value
- copy - copy (see also **scopy**)
- dex - decimal exponentiation (antilog of base 10 logarithm)
- exp - base e exponentiation (antilog of natural logarithm)
- flam - convert F-nu to F-lambda
- fnu - convert F-lambda to F-nu

inv - inverse
 ln - natural logarithm
 log - base 10 logarithm
 lum - convert magnitude to luminosity
 mag - convert luminosity to magnitude
 sqrt - square root

The luminosity to magnitude and magnitude to luminosity operators are based on the standard relation:

$$\text{mag} = -2.5 * \log (\text{lum})$$

where the log is base 10. The F-nu to F-lambda and F-lambda to F-nu operators are based on the relation:

$$\text{F-nu} = \text{F-lambda} * \text{lambda} / \text{nu}$$

where lambda is wavelength and nu is frequency (currently the wavelength is assumed to be Angstroms and so F-lambda is in units of per Angstrom and F-nu is in units of per Hertz). In all the operators it responsibility of user as to the appropriateness of the operator to the input.

The binary operators operate on the spectra in the first operand list and the spectra or numerical constants in the second operand. Numeric constants are equivalent to spectra having the specified value at all pixels. The binary operators are the standard arithmetic ones plus exponentiation and replacement:

replace - replace first operand values by second operand values
 + - addition
 - - subtraction
 * - multiplication
 / - division
 ^ - exponentiation

If the second operand is a spectrum, as mentioned previously, it is interpolated, without flux conservation, to the dispersion function of the first operand spectrum if necessary.

There is a distinction between the first operand and the second operand. The first operand must always be a spectrum. It supplies the dispersion function to be matched by the second operand spectrum. It also supplies a copy of its image header when a new output spectrum is created. In cases where it is desired to have the second operand be the dispersion/header reference and/or the first operand be a the *reverse* parameter is used. For example to subtract a spectrum from the constant 1:

```
cl> sarith 1 - spec invspec reverse+
```

or to subtract two spectra using the subtractand as the dispersion reference:

```
cl> sarith spec1 - spec2 diff reverse+
```

When a binary operation on a pair of spectra is performed the aperture numbers may be required to be the same if *ignoreaps* is no. For images containing multiple spectra the apertures need not be in the same order but only that matching apertures exist. If this parameter is set to yes then aperture numbers are ignored when the operation is performed. For multiple spectra images the second operand spectra are matched by image line number rather than by aperture. If the second operand image has fewer lines, often just one line, then the last line is used repeatedly. This feature allows multiple spectra in the primary operand list to be operated upon by a single spectrum; for example to subtract one spectrum from all spectra in the in a multiple spectrum image.

If it is an error to perform an operation on certain data values, for example division by zero or the square root of a negative number, then the output value is given the value specified by the parameter *errval*.

A log of the operations performed may be printed to the standard output, which may then be redirected if desired, if the *verbose* parameter is set. In the output the last bracketed number is the aperture number of the spectrum.

INPUT/OUTPUT

The arithmetic part of **sarith** is fairly straightforward and intuitive. The selection of input spectra from input images and the placing of output spectra in output images can be more confusing because there are many possibilities. This section concentrates on the topics of the input and output. Since the concepts apply to all of the operators it simplifies things to think in terms of copying input spectra to output spectra; the "copy" operator. Note that the task **scopy** is actually just this case of **sarith** with parameters set for copying. While the discussion here is similar to that in the help for **scopy**, the examples for that task are more focused for illustrating this topic than the **sarith** examples which concentrate more on the arithmetic aspects of the task.

Input spectra are specified by an image list which may include explicit image names, wildcard templates and @files containing image names. The image names may also include image sections such as to select portions of the wavelength coverage. The input images may be either one or two dimensional spectra. One dimensional spectra may be stored in individual one dimensional images or as lines in two (or three) dimensional images. The one dimensional spectra are identified by an aperture number, which must be unique within an image, and a beam number. Two dimensional long slit and three dimensional Fabry-Perot spectra are treated, for the purpose of this task, as a collection of spectra with dispersion either along any axis specified by the DISPAXIS image header parameter or the *dispaxis* package parameter. The aperture and band parameters specify a spatial position. A number of adjacent lines, columns, and bands, specified by the *nsum* package parameter, will be summed to form an aperture spectrum. If number is odd then the aperture/band number refers to the middle and if it is even it refers to the lower of the two middle lines or columns.

In the case of many spectra each stored in separate one dimensional images, the image names may be such that they have a common root name and a four digit aperture number extension. This name syntax is called "onedspec" format. Including such spectra in an input list may be accomplished either with wildcard templates such as

```
name*
name.????.imh
```

where the image type extension ".imh" must be given to complete the template but the actual extension could also be that for an STF type image, or using an @file prepared with the task **names**. To generate this syntax for output images the *format* parameter is set to "onedspec" (this will be discussed further later).

From the input images one may select a range of wavelengths with the *w1* and *w2* parameters and a subset of spectra based on aperture and beam numbers using the *aperture* and *beam* parameters. If the wavelength range is specified as INDEF the full spectra are used without any resampling. If the aperture and beam lists are not specified, an empty list, then all apertures and beams are selected. The lists may be those spectra desired or the complement obtained by prefixing the list with '!'. Only the selected wavelength range and spectra will be operated upon and passed on to the output images.

Specifying a wavelength range is fairly obvious except for the question of pixel sampling. Either the pixels in the specified range are used without resampling or the pixels are resampled to correspond exactly to the requested range. The choice is made with the *rebin* parameter. In the first case the nearest pixels to the specified wavelength endpoints are determined and those pixels and all those in between are used. The dispersion relation is unchanged. In the second case the spectra are reinterpolated to have the specified starting and ending wavelengths with the same number of pixels between those points as in the original spectrum. The reinterpolation is done in either linear or log-linear dispersion. The non-linear dispersion functions are interpolated to a linear dispersion.

Using **sarith** with long slit and Fabry-Perot images provides a and simple type of extraction as opposed to using the **apextract** package. When summing it is often desired to start each aperture after the number of lines summed. To do this specify a step size in the aperture/band list. For example to extract columns 3 to

23 summing every 5 columns you would use an aperture list of "3-23x5" and an *nsum* of 5. If not use the step in the aperture list you would extract the sum of columns 1 to 5, then columns 2 to 6, and so on.

In the special case of subapertures extracted by **apextract**, related apertures are numbered using a modulus; for example apertures 5, 1005, 2005. To allow selecting all related apertures using a aperture number the *apmodulus* parameter is used to specify the modulus factor; 1000 in the above example. This is a feature which should be ignored by most users.

The output list of images may consist of an empty list, a or a list of images matching the input list in number. Note that it is the number of image names that matters and not the number of spectra since there may be any number of spectra in an image. The empty list converts to the same list as the input and is shorthand for replacing the input image with the output image upon completion; therefore it is equivalent to the case of a matching list. If the input consists of just one image then the distinction between a output and a matching list is moot. The interesting distinction is when there is an input list of two or more images. The two cases are then a mapping of many-to-many or many-to-one. Note that it possible to have more complex mappings by repeating the same output name in a matching list provided clobbering, merging, and possibly renumbering is enabled.

In the case of a matching list, spectra from different input images will go to different output images. In the case of a single output image all spectra will go to the same output image. Note that in this discussion an output image when "onedspec" format is specified is actually a root name for possibly many images. However, it should be thought of as a single image from the point of view of image lists.

When mapping many spectra to a single output image, which may have existing spectra if merging, there may be a conflict with repeated aperture numbers. One option is to consecutively renumber the aperture numbers, including any previous spectra in the output image when merging and then continuing with the input spectra in the order in which they are selected. This is specified with the *renumber* parameter which rennumbers beginning with 1.

Another options which may be used independently of renumbering or in conjunction with it is to add an offset as specified by the *offset* parameter. This is last step in determining the output aperture numbers so that if used with the renumber option the final aperture numbers begin with one plus the offset.

It has been mentioned that it is possible to write and add to existing images. If an output image exists an error will be printed unless the *clobber* parameter is set. If clobbering is allowed then the existing output image will be replaced by the new output. Rather than replacing an output image sometimes one wants to replace certain spectra or add new spectra. This is done by selecting the *merge* option. In this case if the output has a spectrum with the same aperture number as the input spectrum it is replaced by the input spectrum. If the input spectrum aperture number is not in the output then the spectrum is added to the output image. To add spectra with the same aperture number and not replace the one in the output use the *renumber* or *offset* options.

EXAMPLES

In addition to the examples in this section there are many examples in the help for **scopy** which illustrate aspects of selecting input spectra and producing various types of output. Those examples are equivalent to using the "copy" operator. The same examples will also apply with other operators where the input spectra are modified arithmetically before being copied to the output images.

I. SIMPLE EXAMPLES

The simple examples use only a single input image and create a output image.

1. Examples of unary operations:

```
cl> sarith example1 mag "" magexample
cl> sarith magexample lum "" example2
cl> sarith example1 log "" logexample
```

Note that a place holder for the second operand is required on the command line which will be ignored.

2. Examples of binary operations using constants:

```
cl> sarith example1 + 1000 example2
cl> sarith example1 - 1000 example2 reverse+
cl> sarith example1 / 1000 example2
cl> sarith example1 ** 2 example2
```

3. Examples of binary operations between spectra with matching apertures:

```
cl> sarith example1 + example2 example3
cl> sarith example1 - example2 example3
```

4. Example of binary operations between spectra with the second image consisting of a single spectrum:

```
cl> sarith example1 / flatspec flatexample1 ignore+ errval=1
```

II. MORE COMPLEX EXAMPLES

5. Unary and constant operations on a list of images:

```
cl> sarith example* fnu "" %example%fnu%
cl> sarith example* + 1000 %example%fnu%
```

6. Binary operations on a list of images using a with matching apertures:

```
cl> sarith example* - skyspec %example%skysub%*
```

7. Selecting apertures to operate upon:

```
cl> sarith example* - skyspec %example%skysub%* aper=1,5,9
```

8. Extract the sum of each 10 columns in a by the central spectrum:

```
cl> nsum = "10"
cl> sarith longslit copy "" longslit.ms aper=5-500x10
longslit[5] --> longslit.ms[5]
longslit[15] --> longslit.ms[15]
longslit[25] --> longslit.ms[25]
...
cl> sarith longslit.ms / longslit.ms[*,25] norm ignore+
longslit.ms[5] / longslit.ms[*,25][245] --> norm[5]
longslit.ms[15] / longslit.ms[*,25][245] --> norm[15]
longslit.ms[25] / longslit.ms[*,25][245] --> norm[25]
...
```

9. In place operations:

```
cl> sarith example* + 1000 example* clobber+
example1[1] + 1000. --> example1[1]
example1[2] + 1000. --> example1[2]
...
example2[1] + 1000. --> example2[1]
example2[2] + 1000. --> example2[2]
...
cl> sarith example* flam "" example* clobber+
example1[1] -- flam --> example1[1]
example1[2] -- flam --> example1[2]
...
example2[1] -- flam --> example2[1]
example2[2] -- flam --> example2[2]
...
cl> sarith example* - skyspec "" clobber+ ignore+
```

```

example1[1] + skyspec[1] --> example1[1]
example1[2] + skyspec[1] --> example1[2]
...
example2[1] + skyspec[1] --> example2[1]
example2[2] + skyspec[1] --> example2[2]
...

```

10. Merging existing spectra with the results of operations:

```
cl> sarith example* / flat "" clobber+ merge+ renum+ ignor+
```

REVISIONS

SARITHV2.11

Previously both w1 and w2 had to be specified to select a band to be used. Now if only one is specified the second endpoint defaults to the first or last pixel.

The noise band in multispec data is only copied from the primary spectrum and not modified. This is a kludge until the noise is handled properly.

SARITHV2.10.3

Additional support for 3D multispec/equispec or spatial spectra has been added. The "bands" parameter allows selecting specific bands and the onedspec output format creates separate images for each selected aperture and band.

SARITHV2.10

This task is new.

SEE ALSO

scopy, splot, imarith, imfunction